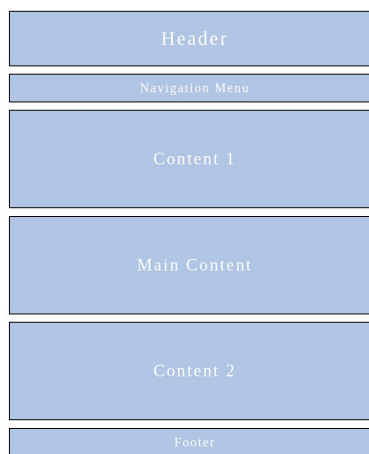


Task 4.4D

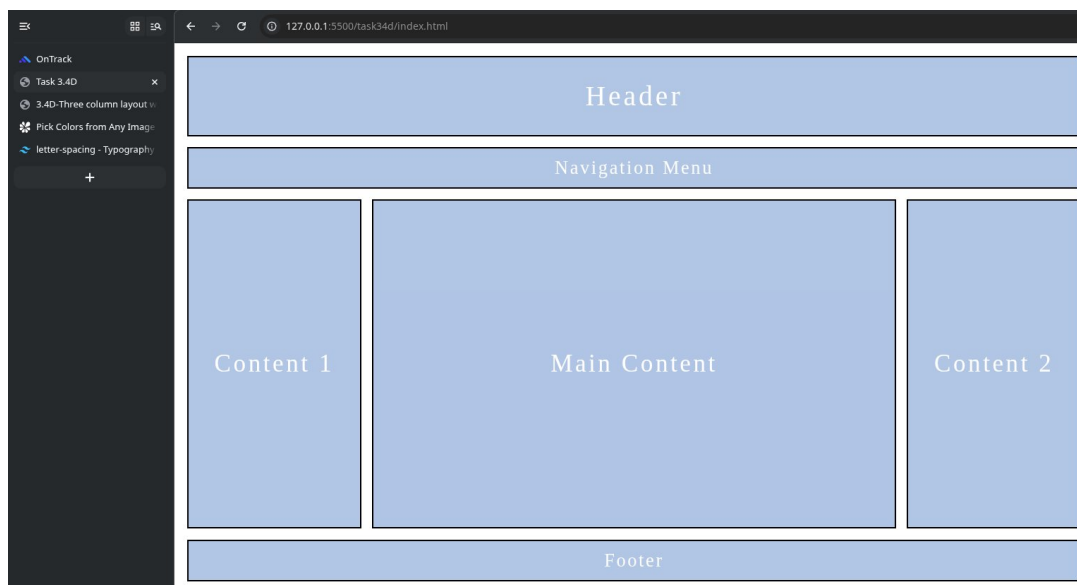
Following the task instructions, I created a three column web page layout with a header, navigation menu, the main contents section (with three columns) followed by a footer at the very end. Since I'm more familiar with Tailwind CSS and it's utility classes oriented approach, I quickly drafted this UI/layout with it to understand the structure, spacing and the responsive behavior. After getting the layout correct, I manually wrote vanilla CSS to replicate an identical layout.

During this conversion, I created my own CSS classes like `.container`, `.box`, `.content-one`, `.main-content`, and `.content-two`. The `.box` class is used for reusable visual styling for the boxes/containers.

For the layout, I stuck with using CSS grids. By default, the page uses `grid-template-columns: 1fr;` which creates one full width column area. Here, `fr` basically means a fraction of the available space. So, `1fr` means one fraction, or all available space if there is only one column. This makes the mobile layout stack vertically.



I then used the `@media screen and (min-width: 768px)` rule to change the layout for larger screens. `768px` is like the conventional value. When the screen is at least `768px` wide, the main section will change to `grid-template-columns: 1fr 3fr 1fr;`. This will create three columns, where the left and right sections take one fraction of the space while the middle main content section takes three fractions, making it wider.



This is mobile first front end development.

Redo

Feedback I received:

Hi Hirusha. This task is not just about replicating the wireframe as per the task sheet, it needs to be filled with real content as per the topic of your choice. As it is a distiction task, I would liek to see proper content in this submission other than a simple wireframe implementation.

Response:

Hi, thanks for the feedback. I'm now resubmitting with real content this time. This page is built based on the content from one of my blog posts about hardening linux, with actual article text, a table of contents and an info sidebar, rather than the bare wireframe.

Mobile View:

An Overview of Linux Hardening for a VPS

Notes on locking down a Debian VPS. This is not a step by step guide, it's just the surface-level ideas worth knowing

[users](#)[ssh](#)[fail2ban](#)[firewall](#)[credentials](#)[trust](#)

On this page

- 1. Base OS & users
- 2. SSH access
- 3. fail2ban
- 4. Auto updates
- 5. Firewall
- 6. Credentials & logs
- 7. Locking down SSH further
- 8. Trust nothing

There's a backstory behind this post but publicly sharing it is not the greatest idea. This covers some interesting hardening ideas to protect your VPS. It's not a step by step guide, the underlying idea is zero trust from the ground up - trust nothing by default, and enable exactly what you've verified.

Article info

Author	Hirusha
Published	2026-02-08
Tags	linux, vps, security, hardening

Quick checklist

- Non-root sudo user
- Root SSH login off
- SSH port moved, keys only
- fail2ban on SSH jail
- Auto security updates
- Default-deny firewall
- Credentials dir, tight perms
- logrotate + offsite backups
- PAM second factor

An Overview of Linux Hardening for a VPS

Notes on locking down a Debian VPS. This is not a step by step guide, it's just the surface-level ideas worth knowing

userssshfail2banfirewallcredentialstrust	
On this page 1. Base OS & users 2. SSH access 3. fail2ban 4. Auto updates 5. Firewall 6. Credentials & logs 7. Locking down SSH further 8. Trust nothing	There's a backstory behind this post but publicly sharing it is not the greatest idea. This covers some interesting hardening ideas to protect your VPS. It's not a step by step guide - more an overview of things worth doing. Everything here is surface level; each point is its own rabbit hole. This is written with a Debian server in mind. If Debian isn't your thing, AlmaLinux is a solid Red Hat based alternative - it even ships with SELinux set to <code>default-deny</code> out of the box. 1. Base OS & users Start by updating everything to the latest version. Create a non-root user and add them to the sudoers file. If a service is running outside Docker, give it its own dedicated account, and set the shell to <code>nologin</code> where possible. Planning to use Docker? Set it up rootless. Set the system's timezone properly, then install and configure <code>haveged</code>. It tops up the entropy pool so the system can generate random data more efficiently and securely. 2. SSH access Disable root login over SSH. Move the daemon off port 22 (and off 2222 - everyone tries that one too) just to fall off automated scanners' radar. For now, only allow public key authentication, and explicitly allow users by username rather than leaving it open. Turn off anything in the SSH config you aren't actually using. 3. fail2ban Set up <code>fail2ban</code> and turn on the SSH jail. You can write custom filters and point it at any other service you run, as long as that service logs to a file. 4. Automatic updates Turning on automatic security updates is generally a good idea. One thing a lot of guides skip: set the correct default security origin, or the updates silently won't apply. If you need 100% uptime this is worth thinking twice about, though the odds of an update actually forcing a restart are low. 5. Firewall Default-deny, then explicitly open the services you want public. Running Docker? Trust the Docker interface fully - restricting it breaks inter-container communication and a lot else. <code>ufw</code> tends to be a bit too simple for that balance, so on Debian reach for <code>iptables</code> directly, or <code>firewalld</code> on Red Hat based systems. If you don't use IPv6, disable it at the firewall (preferred) or via kernel parameters. 6. Credentials & logs Keep credentials in their own directory with tight permissions - a dedicated group or ACLs both work, pick whichever fits. Point applications at files in that directory rather than passing secrets around directly. Outside Docker, set up proper logging with <code>journald</code>, install <code>logrotate</code> so logs don't grow forever, and back up to a server you trust with something like <code>rsync</code>. 7. Locking down SSH further Two options if you want to go further on SSH access specifically: - Static IP: only allow SSH traffic from that one address. - No static IP: put the server on a Tailscale network and restrict SSH to that network in the firewall. This adds Tailscale itself as a new attack surface, but it still beats leaving SSH open to the whole internet. Beyond that, look at pluggable authentication modules (PAM) - Google Authenticator or a YubiKey PAM module - for a second factor on shell access. 8. Trust nothing Whatever you're installing, get it from the correct source if it isn't in Debian's official repos, and validate it. Don't run a random install script off GitHub unless you've actually read through it and understand what it does. The underlying idea is zero trust from the ground up - trust nothing by default, and enable exactly what you've verified.

Article info

AuthorHirusha

Published2026-02-08

Tagslinux, vps, security, hardening

Quick checklist

- Non-root sudo user
- Root SSH login off
- SSH port moved, keys only
- fail2ban on SSH jail
- Auto security updates
- Default-deny firewall
- Credentials dir, tight perms
- logrotate + offsite backups
- PAM second factor